

Q1. Image Negative Transformation

MATLAB Program

```
clc;
clear;
close all;
% Read grayscale image
img = imread('cameraman.tif');
% Convert to grayscale if RGB
if size(img,3)==3
    img = rgb2gray(img);
end
% Image Negative
negative = 255 - img;
% Display Images
figure;
subplot(1,2,1);
imshow(img);
title('Original Image');
subplot(1,2,2);
imshow(negative);
title('Negative Image');
```



Result

The original image and its negative image are displayed.

Interpretation

- Image negative reverses the intensity values.
- Dark regions become bright and bright regions become dark.
- It is useful for enhancing white or gray details embedded in dark regions.

Q2. Histogram Equalization

MATLAB Program

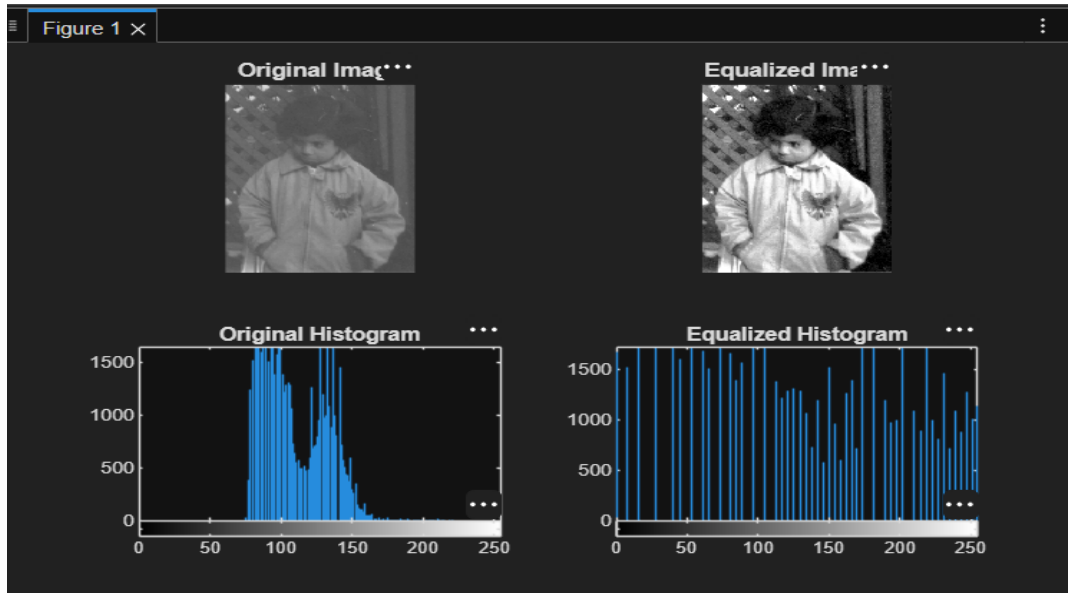
```
clc;
clear;
close all;
% Read image
img = imread('pout.tif');
if size(img,3)==3
img = rgb2gray(img);
end
% Histogram Equalization
eq = histeq(img);
% Display Images
figure;
subplot(2,2,1);
imshow(img);
title('Original Image');
subplot(2,2,2);
imshow(eq);
title('Equalized Image');

subplot(2,2,3);
```

```

imhist(img);
title('Original Histogram');
subplot(2,2,4);
imhist(eq);
title('Equalized Histogram');

```



Result

The original and enhanced images along with their histograms are displayed.

Interpretation

- Histogram equalization spreads the gray levels uniformly.
- Image contrast improves significantly.
- Hidden details become more visible.

Q3. Contrast Stretching

MATLAB Program

```

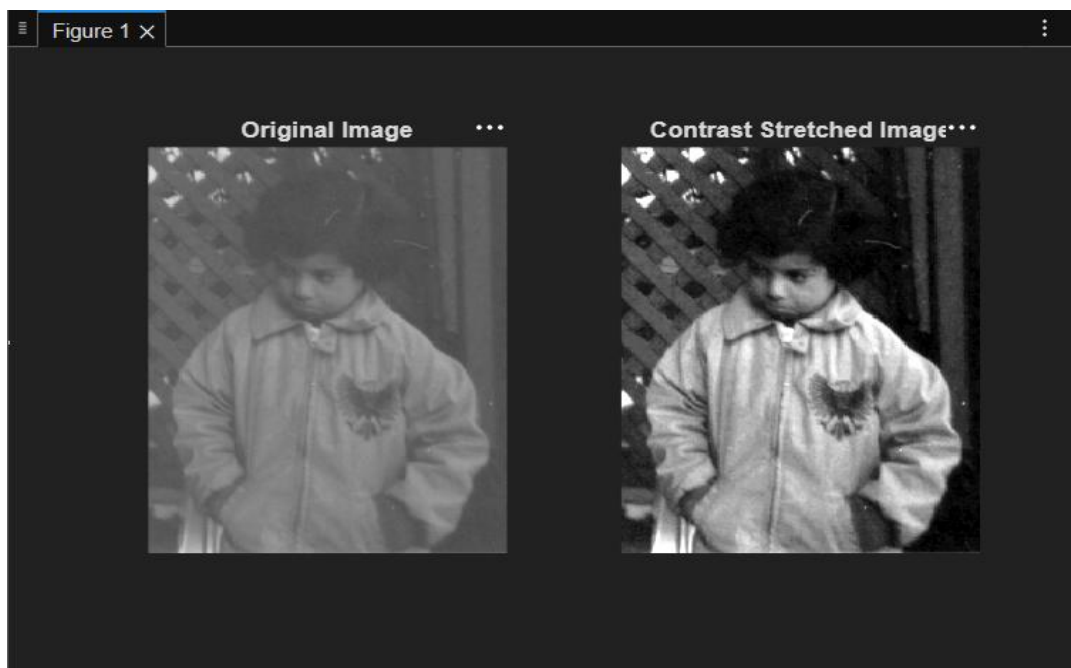
clc;
clear;
close all;
% Read image

```

```

img = imread('pout.tif');
if size(img,3)==3
img = rgb2gray(img);
end
% Contrast Stretching
stretch = imadjust(img, stretchlim(img), []);
% Display Images
figure;
subplot(1,2,1);
imshow(img);
title('Original Image');
subplot(1,2,2);
imshow(stretch);
title('Contrast Stretched Image');

```



Result

The original image and contrast-enhanced image are displayed.

Interpretation

- Contrast stretching increases the intensity range.
- Low-contrast regions become clearer.
- The enhanced image has better visual quality and improved object visibility.

Q4. 3×3 Mean Filter for Image Smoothing

MATLAB Program

```

clc;
clear;
close all;

% Read image
img = imread('cameraman.tif');
if size(img,3)==3
    img = rgb2gray(img);
end

% Add Gaussian noise
noisy = imnoise(img,'gaussian',0,0.01);

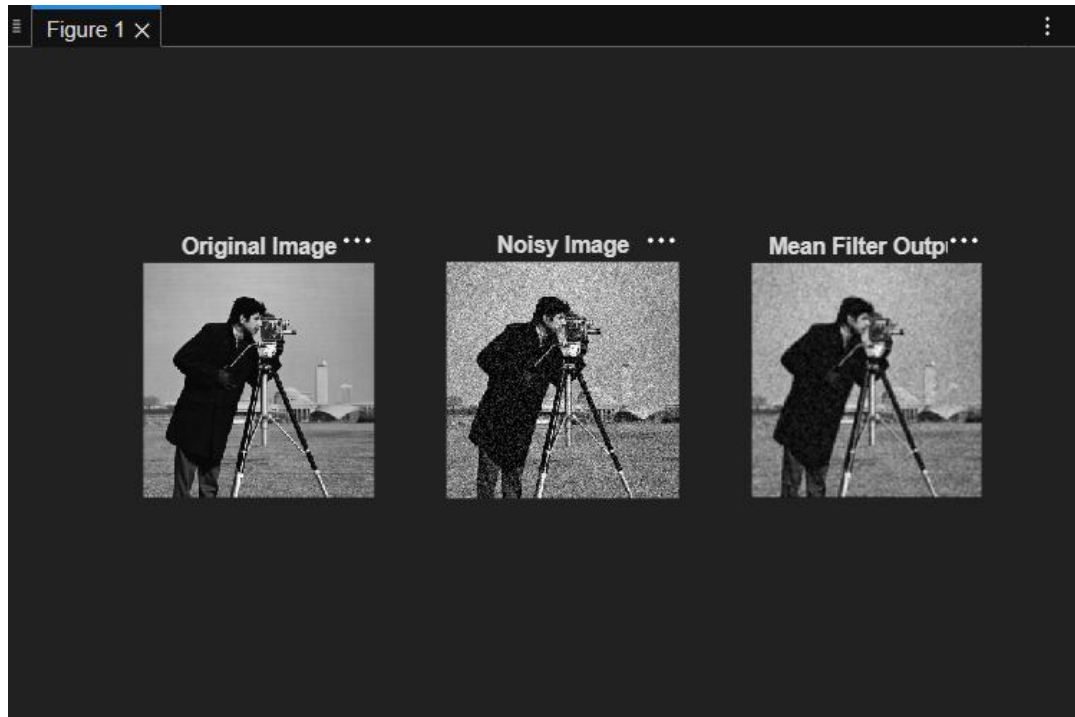
% Create 3x3 Mean Filter
meanFilter = fspecial('average',[3 3]);

% Apply Mean Filter
filtered = imfilter(noisy, meanFilter);

% Display Images
figure;
subplot(1,3,1);
imshow(img);
title('Original Image');

subplot(1,3,2);
imshow(noisy);
title('Noisy Image');
```

```
subplot(1,3,3);
imshow(filtered);
title('Mean Filter Output');
```



Result

The original, noisy, and mean-filtered images are displayed.

Interpretation

- The mean filter smooths the image by averaging neighboring pixels.
- Gaussian noise is reduced effectively.
- However, image edges become slightly blurred because averaging reduces sharp intensity transitions.

Q5. Median Filter for Salt-and-Pepper Noise

MATLAB Program

```
clc;
clear;
close all;
% Read image
img = imread('cameraman.tif');
```

```
if size(img,3)==3
img = rgb2gray(img);
end

% Add Salt & Pepper Noise
noisy = imnoise(img,'salt & pepper',0.05);

% Apply Mean Filter
meanFilter = fspecial('average',[3 3]);
meanImg = imfilter(noisy, meanFilter);

% Apply Median Filter
medianImg = medfilt2(noisy,[3 3]);

% Display Images
figure;
subplot(2,2,1);
imshow(img);
title('Original Image');
subplot(2,2,2);
imshow(noisy);
title('Salt & Pepper Noise');
subplot(2,2,3);
imshow(meanImg);
title('Mean Filter');
subplot(2,2,4);
imshow(medianImg);
title('Median Filter');
```



Result

The original image, noisy image, mean-filtered image, and median-filtered image are displayed.

Interpretation

- The median filter removes salt-and-pepper noise more effectively than the mean filter.
- It preserves image edges while eliminating isolated noisy pixels.
- Therefore, the median filter is more suitable for salt-and-pepper noise removal.

Q6. Laplacian Filtering for Image Sharpening

MATLAB Program

```
clc;
clear;
close all;
% Read image
img = imread('cameraman.tif');
if size(img,3)==3
img = rgb2gray(img);
```



```

end

% Laplacian Filter
lap = fspecial('laplacian',0.2);

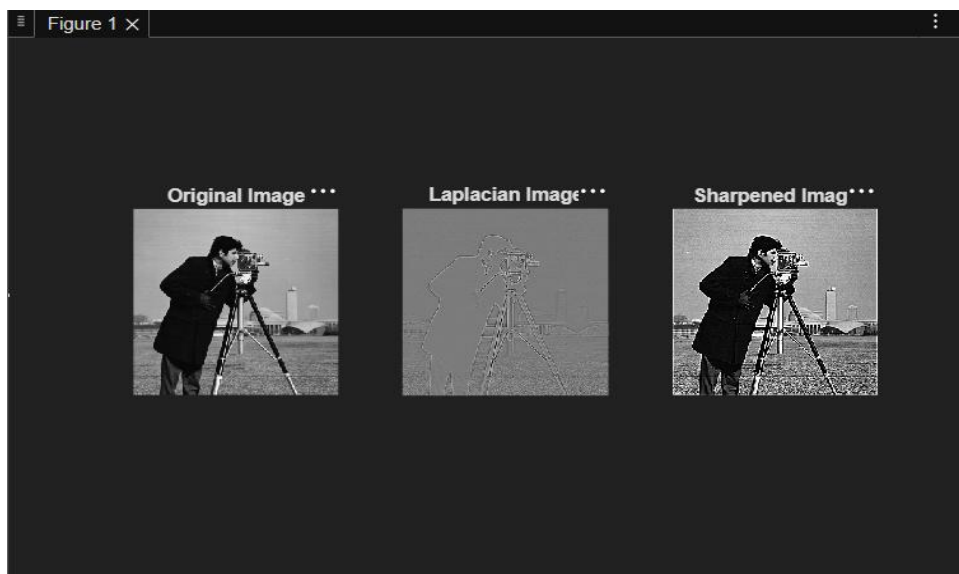
% Apply Laplacian
lapImage = imfilter(double(img), lap);

% Sharpen Image
sharp = double(img) - lapImage;

% Convert to uint8
sharp = uint8(sharp);

% Display Images
figure;
subplot(1,3,1);
imshow(img);
title('Original Image');
subplot(1,3,2);
imshow(mat2gray(lapImage));
title('Laplacian Image');
subplot(1,3,3);
imshow(sharp);
title('Sharpened Image');

```



Result

The original image, Laplacian response, and sharpened image are displayed.

Interpretation

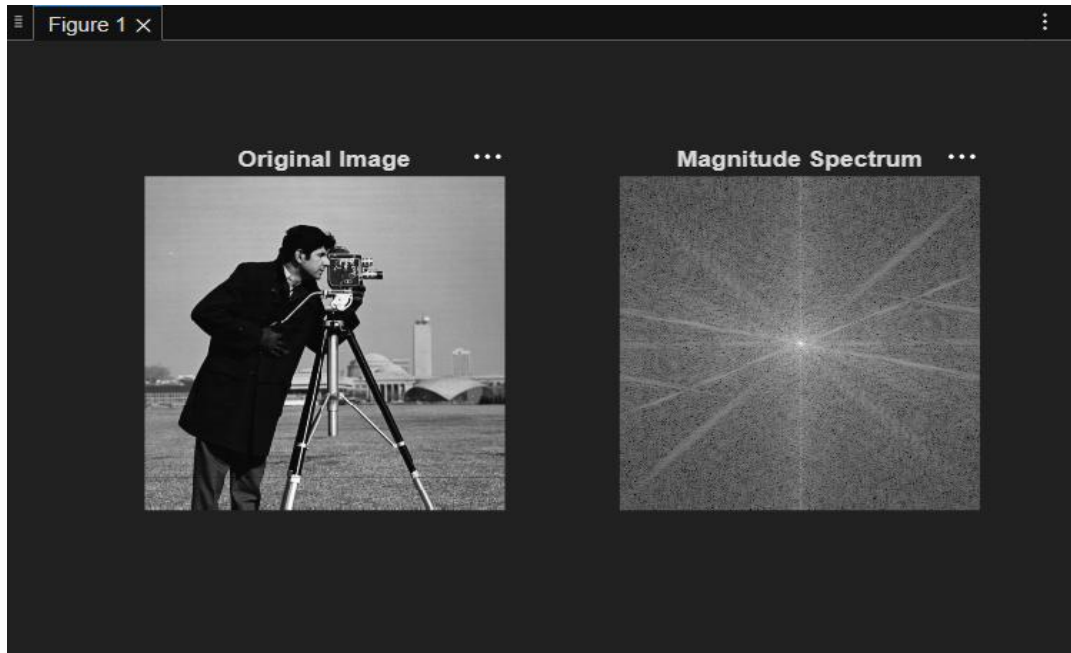
- The Laplacian filter highlights regions of rapid intensity change (edges).
- Subtracting the Laplacian response from the original image enhances fine details.
- The sharpened image appears clearer with more prominent edges.

Q7. Two-Dimensional Discrete Fourier Transform (2D-DFT)

MATLAB Program

```
clc;
clear;
close all;
% Read image
img = imread('cameraman.tif');
if size(img,3)==3
img = rgb2gray(img);
end
% Compute 2D DFT
F = fft2(img);
% Shift zero frequency to center
Fshift = fftshift(F);
% Magnitude Spectrum
magnitude = log(1 + abs(Fshift));
% Display
figure;
subplot(1,2,1);
imshow(img);
title('Original Image');
subplot(1,2,2);
```

```
imshow(magnitude,[]);  
title('Magnitude Spectrum');
```



Result

The original image and its Fourier magnitude spectrum are displayed.

Interpretation

- The center of the spectrum represents low-frequency components.
- The outer regions represent high-frequency components.
- The Fourier Transform helps analyze image information in the frequency domain.

Q8. Low-Pass Filter in the Frequency Domain

MATLAB Program

```
clc;  
clear;  
close all;  
img = imread('cameraman.tif');  
if size(img,3)==3  
img = rgb2gray(img);
```

```

end

[M,N] = size(img);
F = fftshift(fft2(img));

% Low-pass mask
D0 = 50;

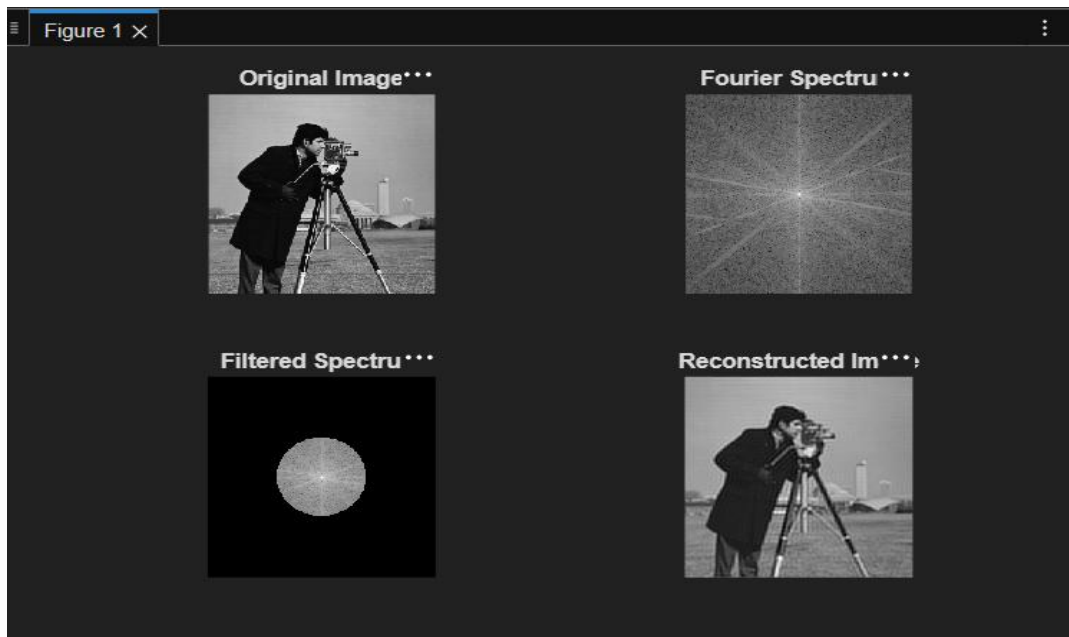
[x,y] = meshgrid(1:N,1:M);
cx = floor(N/2);
cy = floor(M/2);
D = sqrt((x-cx).^2 + (y-cy).^2);
H = double(D <= D0);

% Filtering
G = H .* F;

% Inverse Transform
output = real(ifft2(ifftshift(G)));

% Display
figure;
subplot(2,2,1);
imshow(img);
title('Original Image');
subplot(2,2,2);
imshow(log(1+abs(F)),[]);
title('Fourier Spectrum');
subplot(2,2,3);
imshow(log(1+abs(G)),[]);
title('Filtered Spectrum');
subplot(2,2,4);
imshow(output,[]);
title('Reconstructed Image');

```



Result

The original image, Fourier spectrum, filtered spectrum, and reconstructed image are displayed.

Interpretation

- The low-pass filter removes high-frequency components.
- Noise and fine details are reduced.
- The reconstructed image appears smoother and slightly blurred.

Q9. High-Pass Filter in the Frequency Domain

MATLAB Program

```
clc;
clear;
close all;

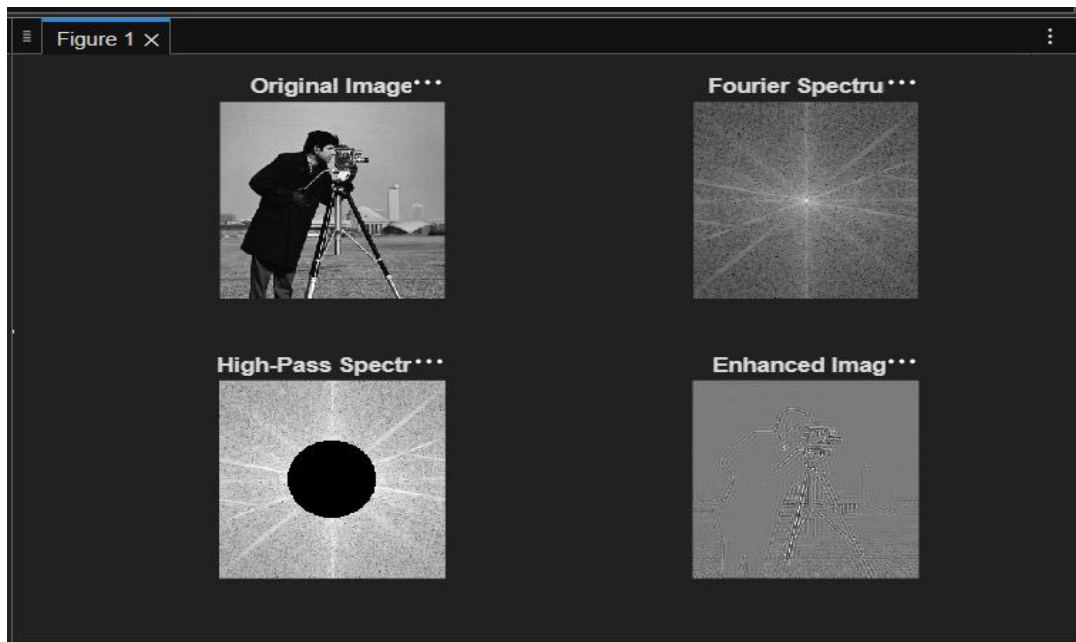
img = imread('cameraman.tif');
if size(img,3)==3
img = rgb2gray(img);
end

[M,N] = size(img);
```

```

F = fftshift(fft2(img));
% High-pass mask
D0 = 50;
[x,y] = meshgrid(1:N,1:M);
cx = floor(N/2);
cy = floor(M/2);
D = sqrt((x-cx).^2 + (y-cy).^2);
H = double(D > D0);
% Filtering
G = H .* F;
% Inverse Transform
output = real(ifft2(ifftshift(G)));
% Display
figure;
subplot(2,2,1);
imshow(img);
title('Original Image');
subplot(2,2,2);
imshow(log(1+abs(F)),[]);
title('Fourier Spectrum');
subplot(2,2,3);
imshow(log(1+abs(G)),[]);
title('High-Pass Spectrum');
subplot(2,2,4);
imshow(output,[]);
title('Enhanced Image');

```



Result

The original image, Fourier spectrum, filtered spectrum, and enhanced image are displayed.

Interpretation

- The high-pass filter suppresses low-frequency components.
- Image edges and fine details become more prominent.
- The output image appears sharper than the original.

Q10. Biomedical Image Enhancement

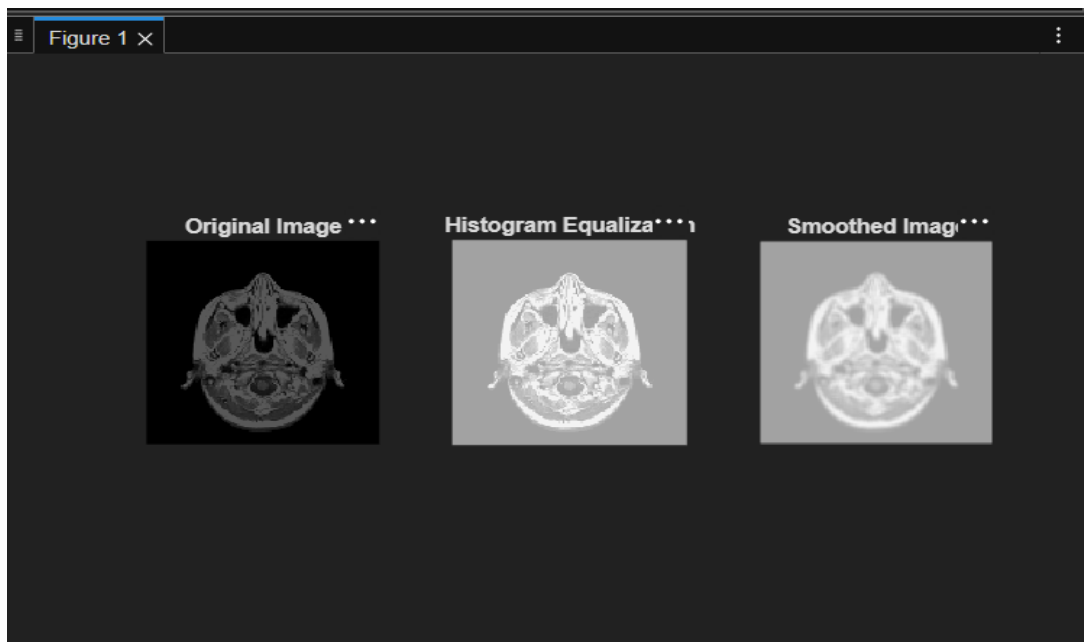
MATLAB Program

```
clc;
clear;
close all;
% Read biomedical image
img = imread('mri.tif');
if size(img,3)==3
img = rgb2gray(img);
end
% Histogram Equalization
```

```

enhanced = histeq(img);
% Mean Filter for smoothing
meanFilter = fspecial('average',[3 3]);
smooth = imfilter(enhanced, meanFilter);
% Display
figure;
subplot(1,3,1);
imshow(img);
title('Original Image');
subplot(1,3,2);
imshow(enhanced);
title('Histogram Equalization');
subplot(1,3,3);
imshow(smooth);
title('Smoothed Image');

```



Result

The original image, histogram-equalized image, and smoothed image are displayed.

Interpretation

- Histogram equalization improves image contrast.
- The smoothing filter reduces random noise.
- The processed image provides better visibility of anatomical structures and overall image quality.